```
function itsComplicated($in): int|string
{
    switch (gettype($in)) {
        case "string":
            return "You gave me a string";
            break;
        case "integer":
            return "You gave me an integer";
            break;
    }

    return 0;
}

if (itsComplicated(new stdClass()) === 0) {
    print "I'm not sure what you gave me, perhaps we'll cover it in a later
    chapter?" . PHP_EOL;
}
```

```
Output:
I'm not sure what you gave me, perhaps we'll cover it in a later chapter?
```

We've removed the parameter type declaration for `$in`, which means that we now need extra code to deal with the uncertainty of how our function might be called. Our function now returns either a `string` or `int`, and we must update the function declaration accordingly. To demonstrate, we pass an object (don't worry, we'll get to those) to `itsComplicated()` and test the output using a strict comparison of `=== 0`.

The function returns `0` because the incoming data isn't `string` or `integer`; therefore, we execute the code in the `if` block. If we hadn't updated the return type of `itsComplicated()` and attempted to return anything other than a `string`, it would have resulted in another fatal `TypeError`.

If you're wondering if the shorthand nullable syntax exists for union types, the answer is a flat "no." Both nullable types and union types were debated at roughly the same time via PHP's RFC documents (the RFC for nullables pre-dates that for unions by about ten months), with nullables making it into the language first. When unions arrived, it was argued (successfully) that using the ? character in a union would